MagicQ miniQ Suite

A collection of DIY hardware and software projects for controlling **ChamSys MagicQ** stage-lighting software with a **Raspberry Pi Pico**, an Art-Net over USB DMX bridge, and a dedicated show PC.

> **Note:** Every project is independent and has its own `platformio.ini` and handbook. The three firmware projects share the same platform (`maxgerhardt/platform-raspberrypi`), board (`rpipico`) and core (`earlephilhower`), so one PlatformIO installation and one toolchain setup is enough for all of them.

Table of contents

- 1. The layout (physical arrangement)
  - 1.1 Projects
  - 1.2 Which project fits which goal
- 2. Hardware overview
- 3. Build & upload (PlatformIO)
- Project structure

1. The layout (physical arrangement)

The suite consists of three Pico firmwares, one show-PC image and two design folders:

```
  [ Enncoder ]        [ Wing ]             [ ArtNet-USB ]        [ Linux-
Installer ]
   2x RP2040 Zero     11 faders            4x DMX ports          Wyse 3040
   (4 encoders ea.)   + 5x13 key grid      Art-Net over USB      Ubuntu +
MagicQ
      |                   |                     |                    (boot-
to-MagicQ)
      +---------+---------+                     |                       |
                |                               |                       |
        [ magicq wing.3mf ]  <---- 3D enclosure + easyEDA/ design files
```

1.1 Projects

Folder	What it does
ArtNet-USB/	Raspberry Pi Pico 4-port **Art-Net ↔ DMX512** bridge over USB. The Pico enumerates as a USB Ethernet (CDC-NCM) adapter on the PC, answers ArtPoll, and drives four DMX ports (GPIO 2–5), each configurable as input or output, with RDM gateway and a config web page (http://10.0.0.10/). PlatformIO / Arduino-Pico.
Enncoder/	**ChamSys MagicQ encoder controller**: two Raspberry Pi Pico (RP2040 Zero) modules — left and right — each with 4 rotary encoders with push switches. The modules emulate an **absolute + relative USB HID mouse** that moves the cursor onto the on-

Folder	What it does
	screen encoders and scrolls/clicks there. Identical firmware and pinout on both sides, separate calibration per module (LittleFS `/ecal.bin`). PlatformIO / Arduino-Pico.
Wing/	**DIY MagicQ Compact Mini Connect Wing**: 11 linear faders (10 playback + grand master) read through a 74HC4067 multiplexer and a 5×13 button matrix (46 keys), all over USB HID (keyboard + absolute mouse for the fader drags) — latency-optimized C++ firmware (PlatformIO / Arduino-Pico).
Linux-Installer/	**Show-PC setup for a Dell Wyse 3040**: Ubuntu + Openbox + MagicQ as a dedicated, boot-to-MagicQ lighting console. One script installs all dependencies, autostart, autologin, boot splashscreen, transparent cursor, USB auto-mount and auto-shutdown.
3D/	3D-printable enclosure (`.3mf`) for a MagicQ control wing.
easyEDA/	EasyEDA schematic / PCB design documents for the hardware.

1.2 Which project fits which goal

Goal	Project
Faders on MagicQ's Full Panel + flash/GO keys	Wing/
Turn / click the MagicQ on-screen encoders	Enncoder/
Feed DMX512 fixtures from the PC over USB	ArtNet-USB/
Boot a small PC straight into MagicQ	Linux-Installer/
Print the enclosure / edit the PCB	3D/, easyEDA/

2. Hardware overview

Peripheral	Used by	Connected to
74HC4067 16-channel analog multiplexer	Wing	GP27 (COM) + GP20–GP22, GP26 (S0–S3), fader wipers on C0–C10
5×13 key matrix (dioded)	Wing	rows GP2–GP6, columns GP7–GP19
Rotary encoders (A/B) + push switches	Enncoder (per module)	A = GP26–GP29, B = GP3–GP6, switches GP15/7/14/8 (identical on both modules)
MAX485 / MAX3485 transceivers	ArtNet-USB	DMX ports on GPIO 2–5 via PIO
Dell Wyse 3040 (eMMC)	Linux-Installer	— (runs MagicQ as show computer)

3. Build & upload (PlatformIO)

Prerequisite: PlatformIO Core installed.

```
# Switch to the project you want to build
cd Wing           # or Enncoder / ArtNet-USB

# Compile the firmware
pio run

# Compile + upload to the Pico (hold the BOOTSEL button before plugging in)
pio run -t upload

# Serial console (debug output)
pio device monitor
```

Important: For upload with the earlephilhower core, the Pico normally only needs to be put into bootloader mode via **BOOTSEL** (hold, plug in USB, release) the first time. Later uploads work directly over USB. The resulting file is at `.pio/build/pico/firmware.uf2` and can also be dragged onto the `RPI-RP2` drive.

> **Note:** the Enncoder project builds **both** modules from the same source — the left module (`pio run -d Enncoder -e pico-left`) and the right module (`pio run -d Enncoder -e pico-right`). `ENC_SIDE` only sets the compiled-in default target positions; the per-module **calibration (`/ecal.bin`)** fully determines the mapping, so both sides work regardless of which env was flashed.

> **Note:** `Wing` and `Enncoder` set `board_build.filesystem_size = 64`. That 64 KB **LittleFS** partition is what their runtime calibrations write to (`/fcal.bin`, `/ecal.bin`) — without it `LittleFS.begin()` fails and nothing is saved. `ArtNet-USB` needs no filesystem.

Project structure

```
miniMQ/
├── README.md              # This overview
├── LICENSE
├── Makefile
├── platformio.ini         # Workspace-level PlatformIO config
├── ArtNet-USB/            # Art-Net over USB -> 4x DMX512 (Pico firmware)
├── Enncoder/              # 2x RP2040 Zero (4 rotary encoders each) ->
MagicQ
├── Wing/                 # 11 faders + 5x13 keys -> MagicQ (Pico firmware)
├── Linux-Installer/       # Dell Wyse 3040 show-PC image setup
├── 3D/                    # 3D-printable enclosure
├── easyEDA/               # EasyEDA schematic / PCB documents
├── scripts/               # Helper scripts
└── .github/               # CI workflows
```